

PROJECT IN ONE LINE

Built a resilient Spring Boot market-data platform that ingests daily candles, computes indicators, and serves secure/observable APIs with reliability controls for third-party quota and payload issues.

WHAT I BUILT

- Java 21 + Spring Boot 4 backend, PostgreSQL + Flyway.
- Ingestion pipeline from Alpha Vantage with retries/backoff/circuit breaker.
- Analytics APIs: RSI, MACD, split-adjusted prices, market calendar, data-quality reports.
- Security controls: API keys, role-based access, rate limiting, daily quotas.
- Ops: structured logs, request IDs, Prometheus metrics, OTLP tracing, runbook + SLO docs.

KEY TECHNICAL DECISIONS

1. Idempotent pipeline runs
 - ``pipeline_run`` tracks run metadata + unique ``idempotency_key``.
 - Safe retries/backfills; auditable outcomes.
2. Quarantine instead of failing the whole run
 - Invalid rows stored in ``ingestion_quarantine`` with reason/payload/run_id.
 - Preserves throughput and debugging visibility.
3. Two quota layers
 - Provider quota (Alpha Vantage) + per-app-key usage limits.
 - Prevents both upstream exhaustion and downstream abuse.
4. Time-series scalability
 - ``price_candle`` partitioned by year on `trade_date`.
 - Indexed symbol/date access for faster range queries.
5. Read-performance optimizations
 - Caching for market calendar and adjusted-price ranges.

SYSTEM SHAPE

Client/UI -> API key auth -> quota/rate limit -> services -> PostgreSQL
-> Alpha Vantage client

CONCRETE SCOPE MARKERS

- 27 mapped API endpoints.
- 8 Flyway migrations.
- Service domains: ingestion, indicators, market, calendar, quality, retention.

HOW I HANDLE FAILURES

- Retryable classification for external failures.
- Circuit breaker opens on repeated failures.
- Partial completion status when some symbols fail.
- 429 responses for rate/quota breaches with limit headers.

TRADEOFFS / WHAT I WOULD IMPROVE

- Persist app-key quotas and generated key metadata for multi-node scale.
- Expand integration testing for idempotency/resilience paths.
- Add richer key lifecycle and audit controls.

90-SECOND TALK TRACK

MarketLens is designed as a reliable pipeline, not just an API wrapper. I focused on operationally safe ingestion under real-world provider constraints: idempotent runs, resilient external calls, row-level quarantine, and clear run status accounting. I paired that with production controls (API security, quotas, observability, runbooks) and data scaling choices like partitioned candles and caching. The result is a practical service that is easier to run, debug, and extend in production.